Report Jungfrau detector calibration

Schwallsunk

June 2026

1 Introduction

For the successful deployment of new x-ray detectors they need to be characterized/calibrated first. To achieve this a well controlled signal needs to be applied to the input of the detector and its output needs to be recorded. These two data sources, together with the assumption of linear behavior of the system, allow for the characterization/calibration of the system. In the given case this well controlled input signal is created in the form of characteristic x-ray produced by x-ray fluoresence. These characteristic x-ray are monochromatic in nature, meaning that they are perfectly centered around a single energy due to the way they are produced in the atoms shell. These x-ray are emitted after a change in the energy level of a shell electron of a material. To produce the vacancy of a single electron in its electron shell x-ray photons are used. These knock out electrons from the atom's shell, leading to a vacancy. Since these energy levels within the atomic electron shells are well defined, the resulting emitted photon energy is also well defined, leading to the monochromatic emitted x-ray. These characteristic x-ray can be produced by different energy level transitions, meaning that we can have multiple characteristic x-ray being emitted at different energies. These in a system are then labeled as for example: K-α, K-β for transitions from the second inner most shell to the inner most shell or for the transition from the third most inner shell to the inner most shell. The same logic applies for the second inner most shell, there its just the K that changes to L.

2 Methods

2.1 Experimental setup

The experimental setup consisted of a 60kV x-ray tube with a strongly collimated beam, a movable sample holder in which the samples were arranged at a 45 degree angle and a Jungfrau detector named Carlos. It looked similar to Fig. 1.
The x-ray tube was used to produce x-ray fluoresence in the metal samples. The samples then emit characteristic x-ray, which are monochromatic. These monochromatic photons impinge on the x-ray detector allowing measurement of the detector system response with a well known input signal.

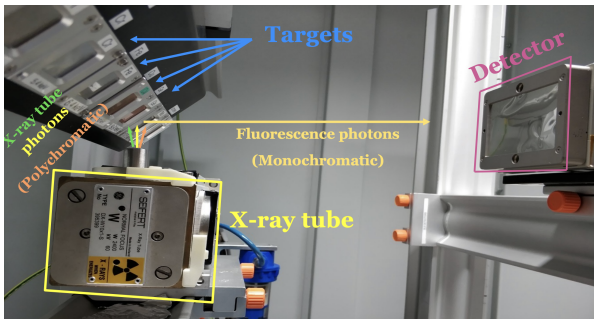


Figure 1: Experimental calibration setup presented in lecture. Image taken from lecture slides.

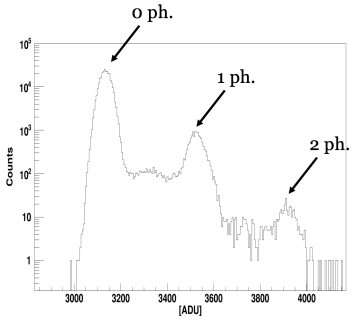


Figure 2: Good aquistion spectrum for a single pixel. Taken from lecture slides

2.2 Material samples

The following material samples were used together with their expected K-α energy emissions:

- Silver $E_{Ag} = 22.2keV$
- Chromium $E_{Cr} = 5.4keV$
- Copper $E_{Cu} = 8keV$
- Iron $E_{Fe} = 6.4keV$
- Indium $E_{In} = 24.2keV$
- Molybdenum $E_{Mo} = 17.5keV$
- Selenium $E_{Se} = 11.2keV$
- Silicon $E_{Si} = 1.74keV$
- Titanium $E_{Ti} = 4.5keV$

2.3 Settings Jungfrau detector "Carlos"

	Material images	Noise images
Number of frames aquired	100	1000
Exposure time per frame [us]	10	10
Aquisition period [ms]	1	1

2.4 Settings x-ray tube

- Tube acceleration voltage: 30kV
- Tube current: 80mA

2.5 Data analysis

The goal was to extract the gain of the system converting the ADUs into keV as well as the equivalent noise charge of the system. To achieve this the 100 frames for each material are binned into the same histogram. This histogram shows a prominent 0-photon peak as well as much less prominent 1-photon peak. There should be even more peaks, characterized by the amount of electron shells of the atom and the incident x-ray tube peak voltage but these not visible in the data taken from these experiments. In theory, both should be connected by a plateau region as can be seen in Fig. 2 and there the added emission lines can also be seen labeled 2 ph.
The 0-photon peak represents the intrinsic noise of the detector. Whereas, the 1-photon peak is the energy expected from the materials K-α emission (the one line we are after for calibration purposes) convoluted with the detector response leading to a gaussian looking peak. The plateau region in between should represent the photons with partial charge cloud deposits in neighboring pixels. Thus, only partial charges are detected within each pixel for a given incident photon.
To get the system gain from these histograms, a linear combination of a Gaussian and a Gaussian with a plateau are fitted onto the given histogram. The fitting is done using "scipy.optimize.curve_fit". As an example, the fitting can be seen in Fig: 3.
The resulting mean value (representing the ADU value of the ADC channel) from the 1-photon peak

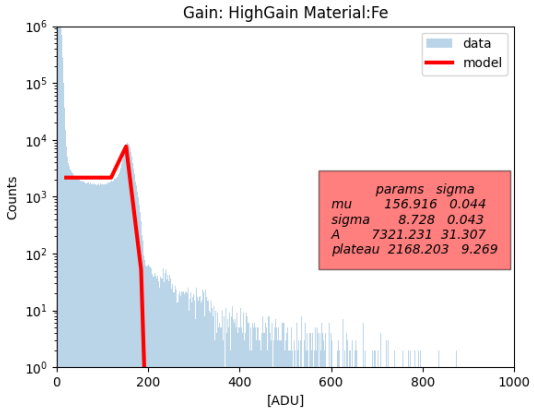


Figure 3: Results of fitting the peaks for silver in high gain mode

(Gaussian with plateau) is used with the according K-α value and plotted in a plot with the K-α energy on its x-axis and the ADU on its y-axis. This point cloud is linear fitted, where the slope corresponds to the gain of the system in [keV/ADU]. As an example the fitting can be seen in Fig: 4b.
For the evaluation of the equivalent noise charge per pixel, dark frames are required. These dark frames are images acquired by the detector at the same settings as the ones used for material measurements. On these dark frames statistics are performed on a pixel wise manner. N acquired dark frames lead to N measurement points for the noise of a single pixel. From this series the standard deviation is taken, resulting in $[\sigma_{noise}] = ADU$. This value is converted using the gain calculated above in the following formula:

$$ENC[e^{-}r.m.s.] = \frac{10^3 \cdot \sigma_{noise}}{Gain \cdot \varepsilon_{pair}} \tag{1}$$

Sidenote: Multiple frames are acquired instead of just increasing the overall exposure time for reasons of potential signal saturation, whilst simultaneously being able to get reasonable count statistics.

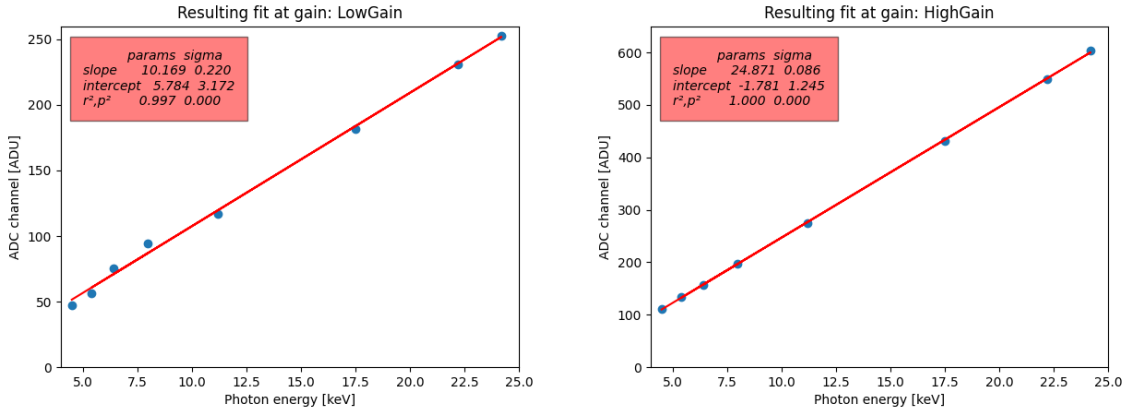
3 Results

3.1 Results of gain analysis:

In Fig. 4a the linear fit to determine the gain of Carlos at its low gain setting can be seen. In Fig. 4b the same can be seen but adjusted to the high gain setting of the system. This leads to the overall following insights:

	Gain [ADU/keV]	Uncertainty (σ) [ADU/keV]
High gain	24.871	0.086
Low gain	10.169	0.22

Table 1: Results gain analysis



(a) low gain

(b) high gain

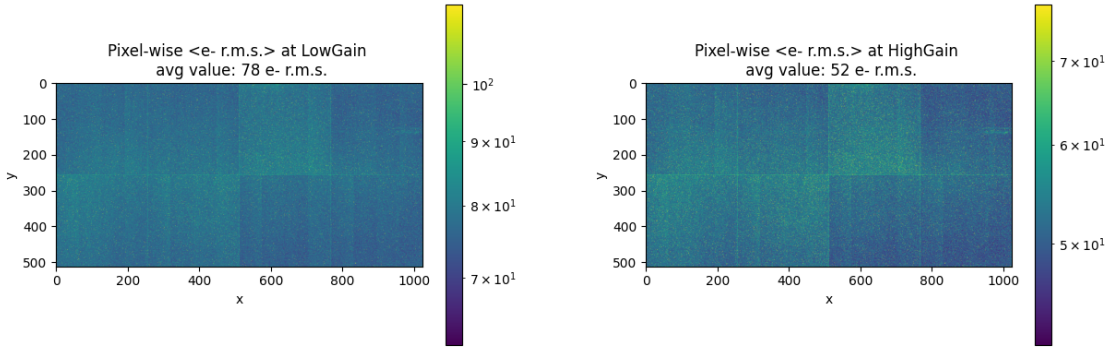
Figure 4: Linear fitted gain results

3.2 Results of noise analysis:

In Fig. 5a the linear fit to determine the gain of Carlos at its low gain setting can be seen. In Fig. 5b the same can be seen but adjusted to the high gain setting of the system. This leads to the overall following insights:

	ENC [e^- r.m.s.]	Uncertainty (σ) [e^- r.m.s.]
High gain	52	5
Low gain	78	6

Table 2: Results noise analysis



(a) low gain (b) high gain

Figure 5: Results of the equivalent noise analysis of the system

4 Discussion

4.1 Discussion gain analysis

Looking at the results Tab.1 the high gain setting delivered a usable result. Looking at the low gain result, there the uncertainties are huge (literally 100%). This means that this result needs to be taken with a big grain of salt. This parameter would benefit a lot from an improved test setup compared with the high gain setting. This could be achieved increasing the overall characteristic photon count at the detector (reduce detector-material distance, increase x ray tube current, increase acquisition time = meaning increasing number of acquired frames) in another experiment.

4.1.1 Computed gain and measured gain

Looking at the values calculated by doing a full detector gain analysis by hand, significant differences can be identified especially for the high gain setting. There the value is considering the given uncertainty is not even within a standard deviation. Looking at the acquisition chain of the system, the

	Measured gain [ADU/keV]	Uncertainty	Calculated gain [ADU/keV]
High gain	24.871	0.086	245.6
Low gain	10.169	0.22	43.3

Table 3: Comparison measured and calculated gain

different gain elements can have different inputs on the system. Looking more in detail into the system design the charge-amplifier stage is designed with proper high fidelity capacitors (MIM capacitors I believe), very costly in terms of ASIC design but precise. Going further down the chain the double sampling system is the next one. This system might introduce some problems but considering that the input of the CDS has a capacitor and the feedback is a capacitor. It could be assumed that those two systems being close to each other and the output of the system being governed by the ratio of both, might very likely be very stable (also considering that robust ASIC design usually includes working with ratios and not with absolute values, and that parts close to each other are usually similarly off from the design spec.) Further down, there is the voltage buffer, which can be assumed as very stable. Then there is the column buffer. And very likely this thing is the culprit we are looking for. According to PSI Jungfrau report it seems like these buffers serve as a line read out system (similar to a CCD sensor readout strategy), whilst also allowing the low power pixels to charge something, whilst the buffer itself being able to drive a high power output, whilst preserving the high sensitivity front end. Meaning that there is a whole lot of things going on potentially skewing the signal. The most likely culprit for this might be the use of capacitors in the process, which due to the different locations of the chip being quite different from each other (border of ASIC vs center) additionally the capactitors used for this read out might be not of a dedicated MIM quality but lower tier and less well controlled in their absolute value, leading to this overall difference in measured vs calculated gain values.

4.2 Discussion noise analysis

Looking at the results given in Tab.2 there is a clear difference in noise between the high and low gain setting. The values calculated make sense considering the fact, that the high gain stage is more sensitive to a smaller overall integrated deposited charge in the detector.

4.2.1 Required calibration data for a per pixel calibration

To successfully perform a per pixel calibration, we would need to achieve a similar amount of counts which we accumulated by aggregating all pixel measurements into a single histogram. This means that the amount of data that was acquired is equal to the data for a single pixel, assuming that the given histogram is considered good enough for a single calibration. Meaning the needed amount of data would skyrocket by a factor of $512 \cdot 1024 = 524288$.

5 Analysis python code

```
import requests
import numpy as np
import matplotlib.pyplot as plt
import time
import os
import pandas as pd
from scipy.optimize import curve_fit
from scipy import stats
from scipy import special
from matplotlib.colors import LogNorm



# Define constants, camel case is neglected
gainz = ["LowGain","HighGain"]

properties_phyiscal = {"materials":["Ag","Cr","Cu","Fe","In","Mo","Se","
    Ti"],"energy":[22.2,5.4,8,6.4,24.2,17.5,11.2,4.5]}
initial_guesses_mu={"LowGain":[210,50,100,100,220,200,100,50],"HighGain"
    :[500,150,200,150,600,400,250,100]}
initial_guesses_sigma={"LowGain":[3,0.5,0.5,1,3,3,1,1],"HighGain"
    :[5,3,3,3,5,5,5,3]}
initial_guesses_plateau={"LowGain"
    :[100,1000,1000,1000,100,300,1000,1000],"HighGain"
    :[100,500,800,800,80,100,500,800]}
ADUs_bimodal = {"LowGain":[],"HighGain":[]}
n_bins =  16383 # Dut to 14 bit ADC
path_to_storage = r'E:\\detecting_the_unknown\\results_plateau\\'


# This is the header with various meta data
# mostly used for debugging
# taken from the provided jupyter notebook as is
header_dt = np.dtype(
    [
        ("Frame-Number", "u8"),
        ("SubFrame-Number/ExpLength", "u4"),
        ("Packet-Number", "u4"),
        ("Bunch-ID", "u8"),
        ("Timestamp", "u8"),
        ("Module-Id", "u2"),
        ("Row", "u2"),
        ("Column", "u2"),
        ("Reserved", "u2"),
```

```
        ("Debug", "u4"),
        ("Round-Robin-Number", "u2"),
        ("Detector-Type", "u1"),
        ("Header-Version", "u1"),
        ("Packets-caught-mask", "8u8")
    ]
)

# The data type describing the file is created by taking the header and
    then
# adding a 512x1024 field of unsigned 16 bit integers
# taken from the provided jupyter notebook as is
# Important note: Data is effectively 16 bit long.
# This means there are two bits added to every pixel describing the gain
    setting of the pixel. Not just a type cast mismatch.
# We need to get rid of them by right shifting the data
raw_dt = np.dtype([('header', header_dt), ('images', np.uint16,
    (512,1024))])

# Then we can package up the reading and decoding in a function
# normally you would read from disk but it is equally easy to fetch
# data from the internet
# taken from the provided jupyter notebook as is
def download(fname, n_frames = 100):
    t0 = time.perf_counter()
    #replace with group 2 for second group
    base_url = 'https://drive.switch.ch/index.php/s/??????/download?path
        =%2Fgroup2&files={}' # Change fetch URL according to your given
        URL
    url = base_url.format(fname)
    print(f'fetching:-{url}-—-', end = '')
    response = requests.get(url, stream=True)


    data = np.zeros(n_frames, dtype = raw_dt)
    for i,chunk in enumerate(response.iter_content(chunk_size=raw_dt.
        itemsize)):
        data[i] = np.frombuffer(chunk, dtype = raw_dt)
        if i==n_frames - 1:
            break


    t = time.perf_counter()-t0
    print(f'{t:.2f}s')
    return data
```

```
# Function describing fitted gauss peak
def gauss(x, mu, sigma, A):
    return A*np.exp(-(x-mu)**2/2/sigma**2)

def plateau_gauss(x, mu, sigma, A, plateau):
    return A*np.exp(-(x-mu)**2/2/sigma**2)+plateau*(1-special.erf((x-mu)
        /(sigma*np.sqrt(2))))/2


#In case single fitting wants to be done and splitting of every single
    histo
def enc_per_pixel(N_pixel,inital_guess_mu, initial_guess_sigma,
    gain_global):
    y,x,_=plt.hist(N_pixel, bins=n_bins, alpha=.3, label='data')
    x=(x[1:]+x[:-1])/2
    expected = (inital_guess_mu, initial_guess_sigma, 1) # First guesses
        for gaussian fit
    result = np.zeros_like(y, dtype=float) # create array with zeros for
        hist
    np.log10(y, where=(y > 0), out=result) # take logarithm of all values
         larger than 0 to prevent inf runaway
    params, cov = curve_fit(gauss, x, result, expected) # fit on
        lograithmed data => simplifies the peak detection for the
        algorithm massivel
    sig_noise=params[1]
    return (1000*sig_noise/(gain_global)/3.62)

# Main loop for the fitting of the bimodals on all given raw data
for gain in gainz:
    for i, material in enumerate(properties_phyiscal["materials"]):
        try:
            data = download('T-{}-{}_d0_f0_0.raw'.format(material,gain))
                # download corresponding data set
            current_settings = "Gain:-{}-Material:{}".format(gain,
                material) # create label for plts
            data['images'] = [np.right_shift(image,2) for image in data['
                images'][:]] # getting rid of gain setting bits which are
                not used
            mean_image = data['images'].mean(axis = 0)
            data['images'] = [np.absolute(image-mean_image.astype(int))
                for image in data['images'][:]] # get rid of background by
                 subtracting the mean value of each single pixel across
                the aqusition series, important to take abs val
            real_frames=data['images'].flatten() # convert 2D arrays into
                 1D arrays for histogram production
            plt.close() # Make  plt canvas pristine
```

```
y,x,_=plt.hist(real_frames, bins=np.arange(n_bins), alpha=.3,
    label='data') # Plot histogram
y=y[20:]
x=x[20:]
plt.ylim(1, 1e6) # y axis limits
plt.yscale('log') # log scale
plt.xlim(0, 1000)
x=(x[1:]+x[:-1])/2 # make it centered
#x, y inputs can be lists or 1D numpy arrays
expected = (initial_guesses_mu[gain][i],
    initial_guesses_sigma[gain][i], initial_guesses_plateau[
    gain][i]*2, initial_guesses_plateau[gain][i]) # First
    guesses for gaussian plateau fit
try:
    result = np.zeros_like(y, dtype=float) # create array
        with zeros for hist
    np.log10(y, where=(y > 0), out=result) # take logarithm
        of all values larger than 0 to prevent inf runaway
    x_centers = x[:-1] + np.diff(x) / 2
    params, cov = curve_fit(plateau_gauss, x_centers, result,
         expected,bounds=(0, np.inf)) # fit on lograithmed
        data => simplifies the peak detection for the
        algorithm massively
    sigma=np.sqrt(np.diag(cov)) # get sigma of each gaussian
        from covariance
    x_fit = np.linspace(x.min(), x.max(), 2000) # create x
        coords for plotting fit
    plt.plot(x_fit, 10**plateau_gauss(x_fit, *params), color=
        'red', lw=3, label='model') # plot full bimodal
except: # take care in case of non convergence in log space
    to switch back into normal space for second fitting try in
     case
    params, cov = curve_fit(plateau_gauss, x, y, expected)
    sigma=np.sqrt(np.diag(cov)) # get sigma of each gaussian
        from covariance
    x_fit = np.linspace(x.min(), x.max(), 500)
    #plot combined...
    plt.plot(x_fit, plateau_gauss(x_fit, *params), color='red
        ', lw=3, label='model')
plt.legend()
plt.ylabel("Counts")
plt.xlabel("[ADU]")
plt.title(current_settings)
print(pd.DataFrame(data={'params': params, 'sigma': sigma},
    index=plateau_gauss.__code__.co_varnames[1:]))
```

```
            results=pd.DataFrame(data={'params': params, 'sigma': sigma},
                 index=plateau_gauss.__code__.co_varnames[1:])
            results_rounded=results.round(decimals=3)
            plt.text(600, 100, results_rounded, style='italic', bbox={'
                facecolor': 'red', 'alpha': 0.5, 'pad': 10})
            path_fig=os.path.join(path_to_storage,"results_bimodal_{}_{}.
                png".format(material,gain))
            plt.savefig(path_fig)
            ADUs_bimodal[gain].append(float(results["params"].iloc[0]))

        except Exception as e:
            print("An exception occurred: {}".format(e))
print(ADUs_bimodal)


for gain in gainz:
    plt.close()
    x_points =[]
    y_points =[]
    for i, material in enumerate(properties_phyiscal["materials"]):
        x_points.append(ADUs_bimodal[gain][i])
        y_points.append(properties_phyiscal["energy"][i])
    plt.title("Resulting fit at gain: {}".format(gain))
    plt.xlabel("Photon energy [keV]")
    plt.ylabel("ADC channel [ADU]")
    plt.xlim(4,25)


    res = stats.linregress(y_points, x_points)
    plt.plot(y_points, x_points, 'o', label='experimental data')
    plt.plot(y_points, [(res.intercept + res.slope*y) for y in y_points],
        'r', label='fitted line')
    df_regr = pd.DataFrame(data={"params":[res.slope, res.intercept, res.
        rvalue**2],"sigma":[res.stderr, res.intercept_stderr, res.pvalue
        **2]}, index=["slope", "intercept", "r  , p  "])
    if gain =="HighGain":
        plt.ylim(0,650)
        plt.text(5, 500, df_regr.round(decimals=3), style='italic', bbox
            ={'facecolor': 'red', 'alpha': 0.5, 'pad': 10})
    else:
        plt.ylim(0,260)
        plt.text(5, 200, df_regr.round(decimals=3), style='italic', bbox
            ={'facecolor': 'red', 'alpha': 0.5, 'pad': 10})
```

```
path_fig=os.path.join(path_to_storage,"results_fitting_{}.png".format
    (gain))
plt.savefig(path_fig)
# Do rms analysis of dark frames
data = download('noise_floor_{}_d0_f0_0.raw'.format(gain),n_frames
    =1000) # fetch dark data
data['images'] = [np.right_shift(image,2) for image in data['images'
    ][:]]
pd_rms=data['images'].std(axis = 0)
factor = 1000/3.62/res.slope
enc = pd_rms*factor
#covert measured ADU in energy first and then into electron noise
    using the pair creation energy
plt.close() # Make  plt canvas pristine
fig, ax = plt.subplots()
ax.set_title('Pixel-wise <e- r.m.s.> at {} \n avg value: {} e- r.m.s.
    '.format(gain, int(np.mean(enc))))
ax.set_xlabel('x')
ax.set_ylabel('y')
im = ax.imshow(enc,norm=LogNorm(vmin=np.median(enc)*0.8, vmax=1.5*np.
    median(enc)))
fig.colorbar(im)
path_fig=os.path.join(path_to_storage,"results_enc_{}.png".format(
    gain))
print(gain)
print("standard deviation of enc")
print(int(np.std(enc)))
plt.savefig(path_fig)
```